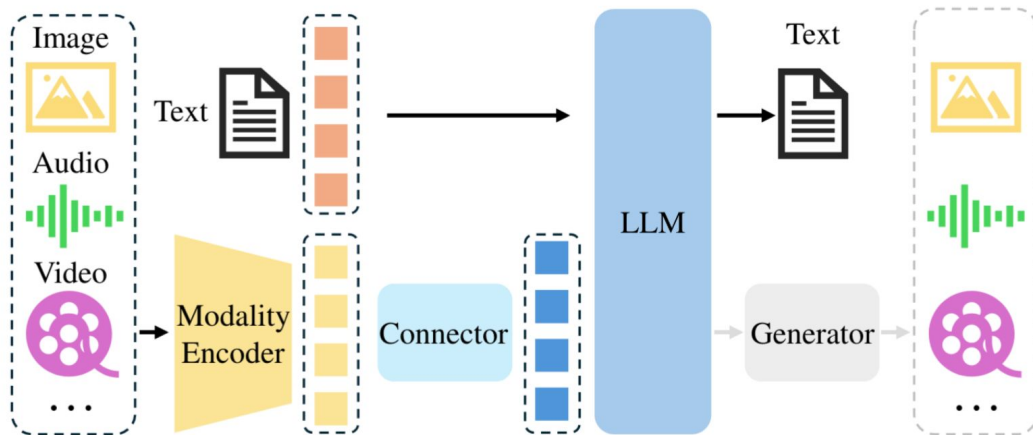


Transformers in Deep Learning

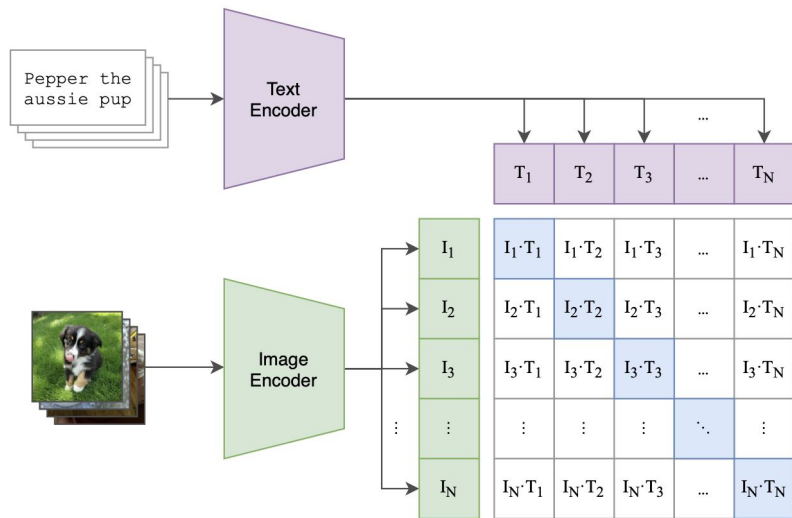
Lecture 6

Lecture 5 Recap

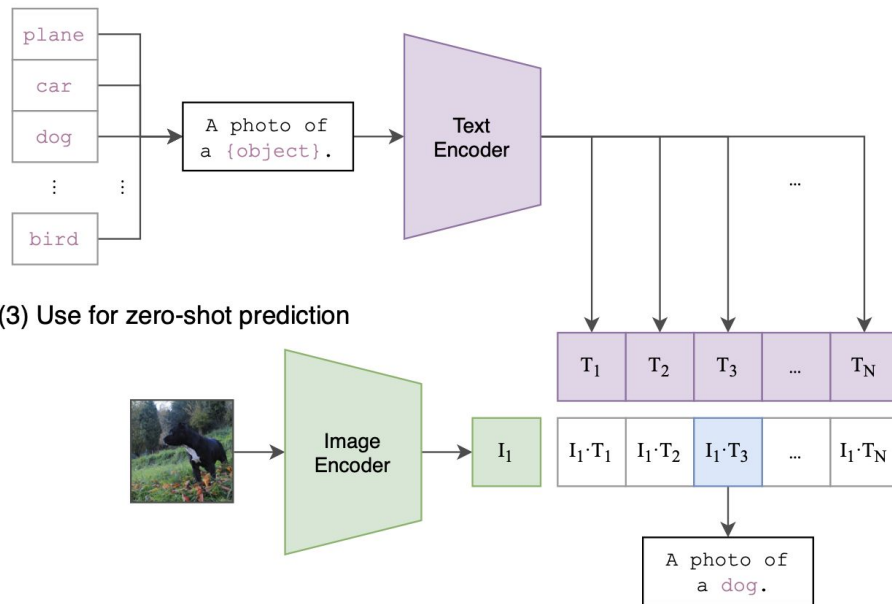
- Use world knowledge and reasoning from LLMs
- Explore various applications that emerge at the intersection of the modalities
- Boost quality of models using patterns in multimodal data that don't exist in unimodal data



Lecture 5 Recap



(2) Create dataset classifier from label text



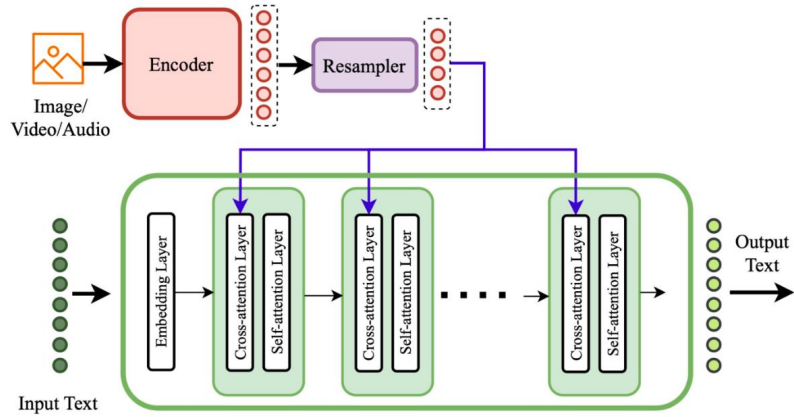
Lecture 5 Recap

Multimodal models can be classified in 2 main types (4 subtypes)
based on the **fusion of input modalities**

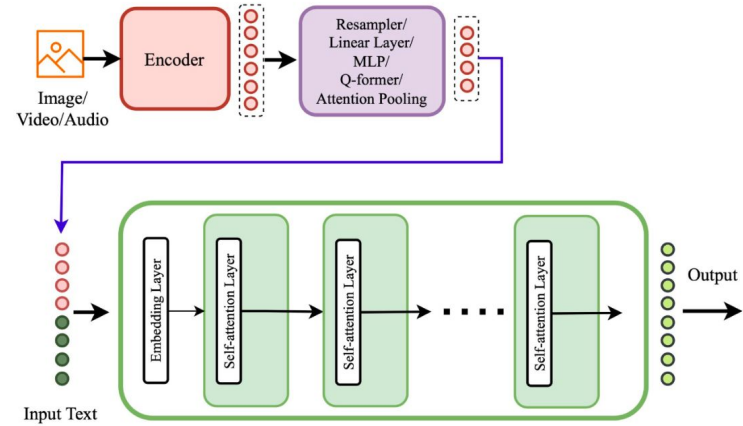


Lecture 5 Recap

Late Fusion



Early Fusion



Transformers Training

Basics

Tokenization
Architecture
Loss function
Optimizer
Learning rate

Systems

Kernels
Parallelism
Quantization
Activation checkpointing
CPU offloading
Inference

Scaling laws

Scaling sequence
Model complexity
Loss metric
Parametric form

Data

Evaluation
Curation
Transformation
Filtering
Deduplication
Mixing

Alignment

Supervised fine-tuning
Reinforcement learning
Preference data
Synthetic data
Verifiers

NN Training basics

- Optimizer (e.g., AdamW, Muon, SOAP) [\[Kingma+ 2014\]](#)[\[Loshchilov+ 2017\]](#)[\[Keller 2024\]](#)[\[Vyas+ 2024\]](#)
- Learning rate schedule (e.g., cosine, WSD) [\[Loshchilov+ 2016\]](#)[\[Hu+ 2024\]](#)
- Batch size (e.g., critical batch size) [\[McCandlish+ 2018\]](#)
- Regularization (e.g., dropout, weight decay)
- Hyperparameters (number of heads, hidden dimension): grid search

Check this to refresh NN Training Tips and Tricks:

<https://www.lri.fr/~gcharpia/deeppractice/2020/tips.pdf>

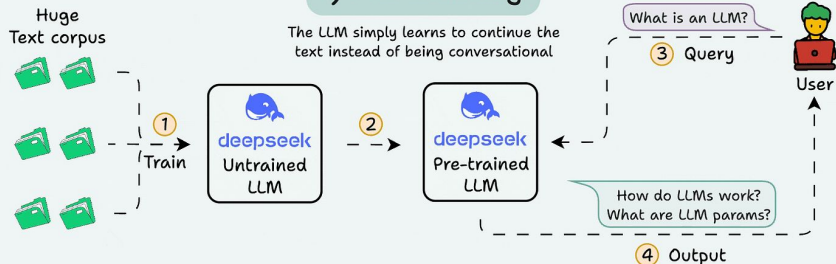
Optimization Settings in Popular LLMs (2023 edition)

Model	Batch Size (#tokens)	Learning Rate	Warmup	Decay Method	Optimizer	Precision Type	Weight Decay	Grad Clip	Dropout
GPT3 (175B)	32K→3.2M	6×10^{-5}	yes	cosine decay to 10%	Adam	FP16	0.1	1.0	-
PanGu- α (200B)	-	2×10^{-5}	-	-	Adam	-	0.1	-	-
OPT (175B)	2M	1.2×10^{-4}	yes	manual decay	AdamW	FP16	0.1	-	0.1
PaLM (540B)	1M→4M	1×10^{-2}	no	inverse square root	Adafactor	BF16	lr^2	1.0	0.1
BLOOM (176B)	4M	6×10^{-5}	yes	cosine decay to 10%	Adam	BF16	0.1	1.0	0.0
MT-NLG (530B)	64 K→3.75M	5×10^{-5}	yes	cosine decay to 10%	Adam	BF16	0.1	1.0	-
Gopher (280B)	3M→6M	4×10^{-5}	yes	cosine decay to 10%	Adam	BF16	-	1.0	-
Chinchilla (70B)	1.5M→3M	1×10^{-4}	yes	cosine decay to 10%	AdamW	BF16	-	-	-
Galactica (120B)	2M	7×10^{-6}	yes	linear decay to 10%	AdamW	-	0.1	1.0	0.1
LaMDA (137B)	256K	-	-	-	-	BF16	-	-	-
Jurassic-1 (178B)	32 K→3.2M	6×10^{-5}	yes	-	-	-	-	-	-
LLaMA (65B)	4M	1.5×10^{-4}	yes	cosine decay to 10%	AdamW	-	0.1	1.0	-
LLaMA 2 (70B)	4M	1.5×10^{-4}	yes	cosine decay to 10%	AdamW	-	0.1	1.0	-
Falcon (40B)	2M	1.85×10^{-4}	yes	cosine decay to 10%	AdamW	BF16	0.1	-	-
GLM (130B)	0.4M→8.25M	8×10^{-5}	yes	cosine decay to 10%	AdamW	FP16	0.1	1.0	0.1
T5 (11B)	64K	1×10^{-2}	no	inverse square root	AdaFactor	-	-	-	0.1
ERNIE 3.0 Titan (260B)	-	1×10^{-4}	-	-	Adam	FP16	0.1	1.0	-
PanGu- Σ (1.085T)	0.5M	2×10^{-5}	yes	-	Adam	FP16	-	-	-

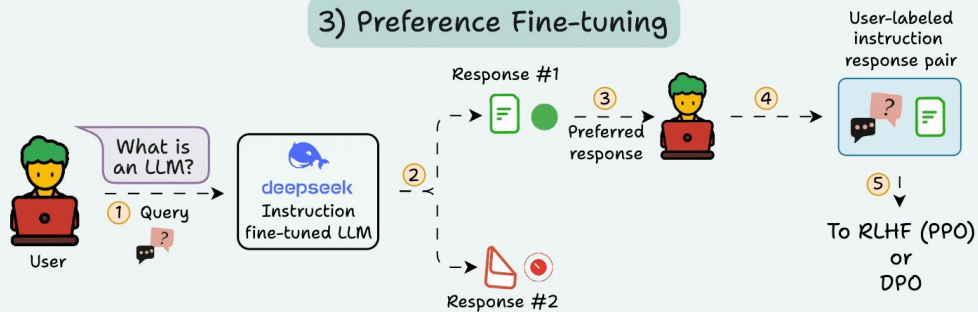
Transformers Training Stages

1) Pre-training

The LLM simply learns to continue the text instead of being conversational

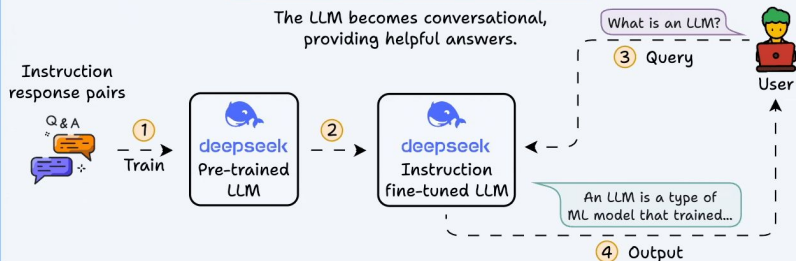


3) Preference Fine-tuning



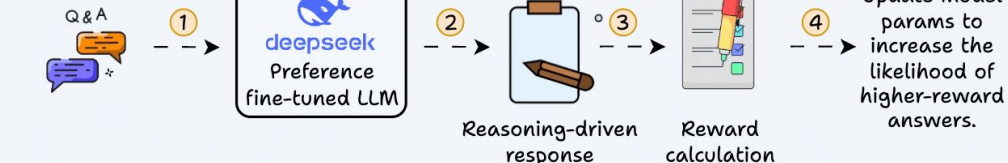
2) Instruction fine-tuning

The LLM becomes conversational, providing helpful answers.

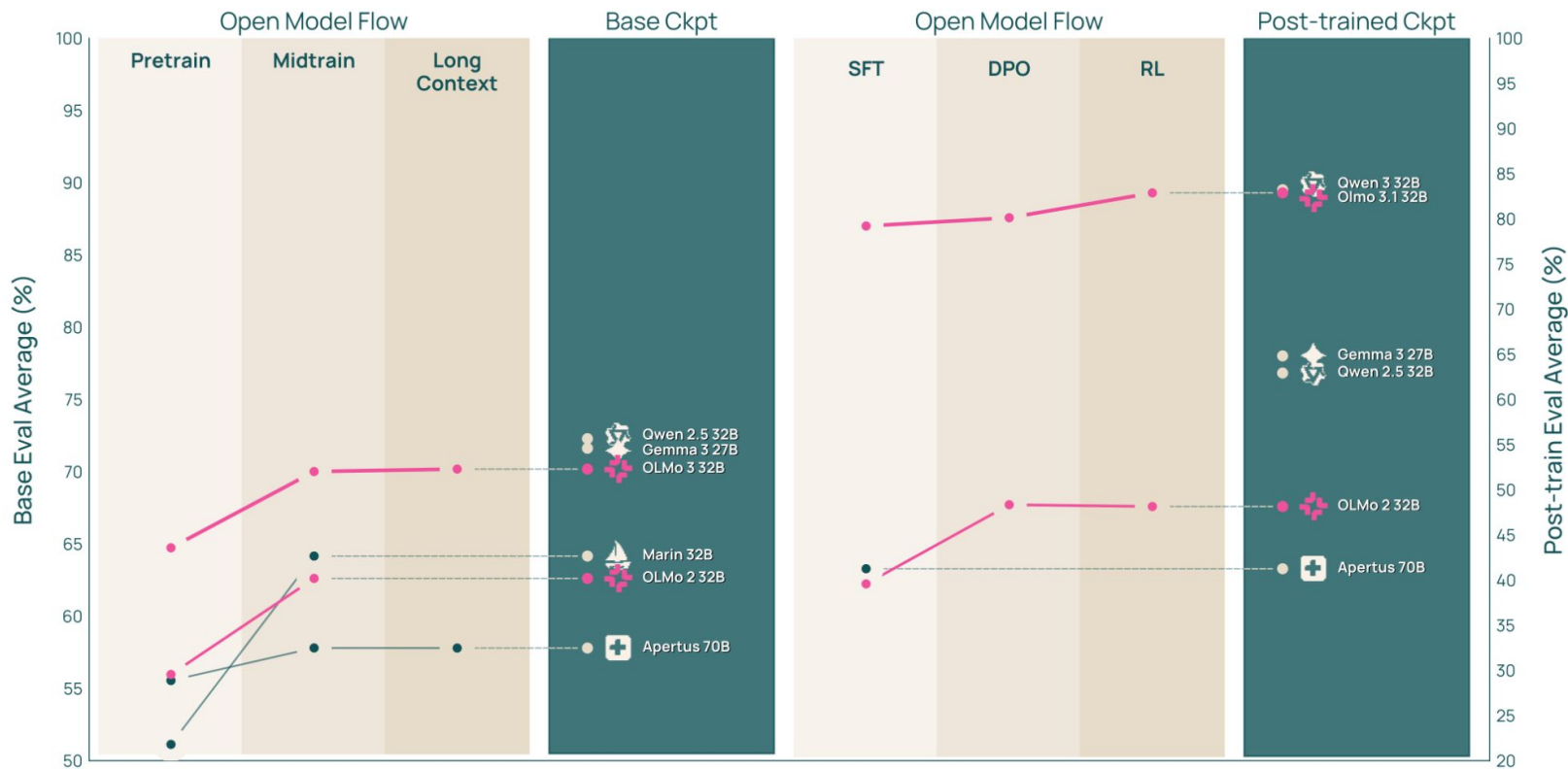


4) Reasoning fine-tuning

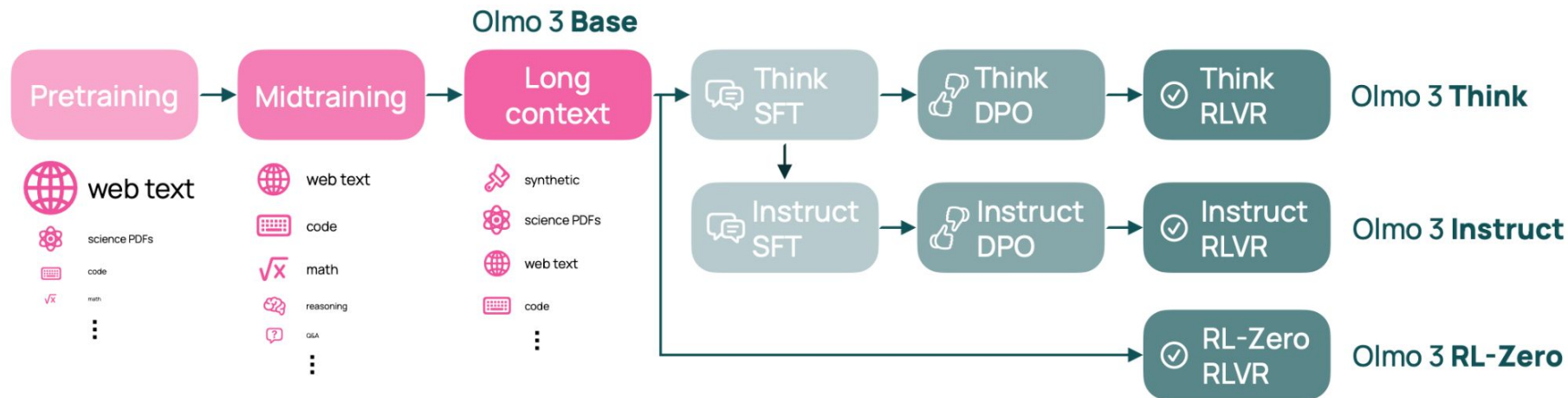
Reasoning task with a definitive answer



Transformers Training Stages



Transformers Training Stages



0) Randomly initialized LLM



User

What is
an LLM?

① Query

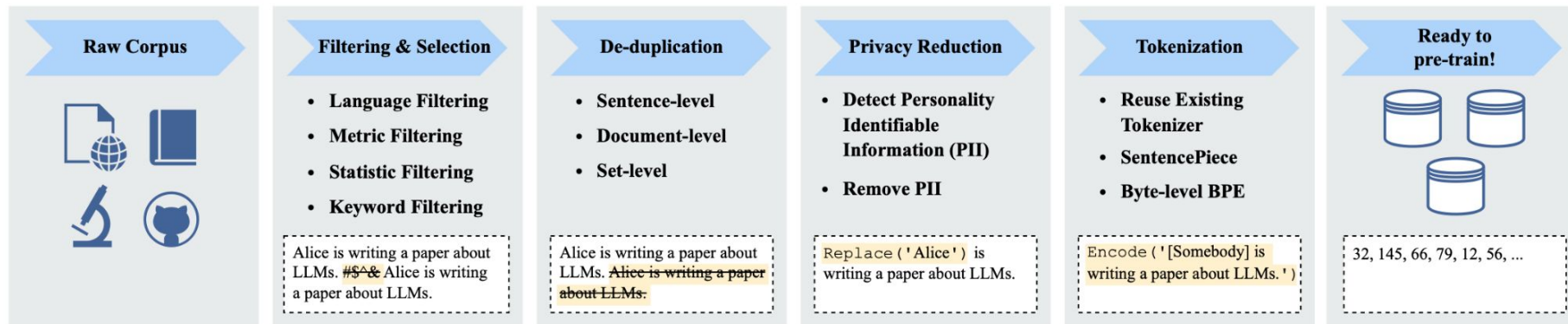


② Output

try peter
hand and
hello 4485n

Random output

Data Preprocessing



Data Preprocessing

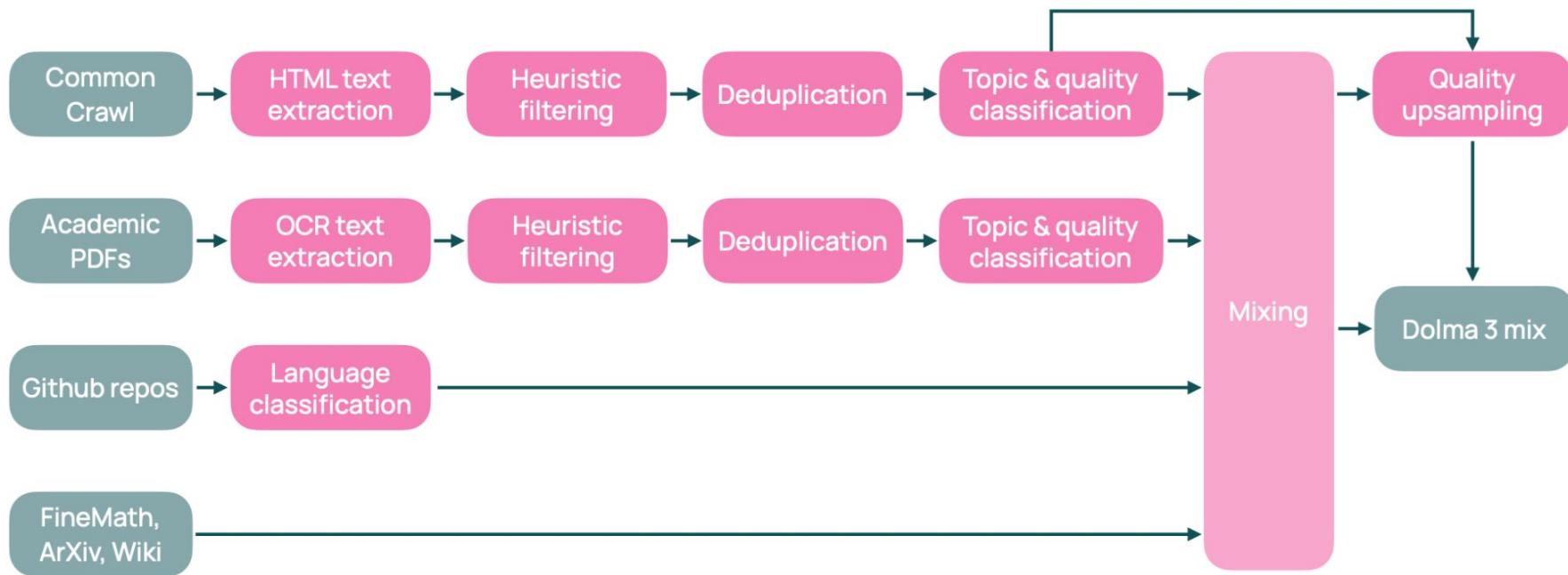
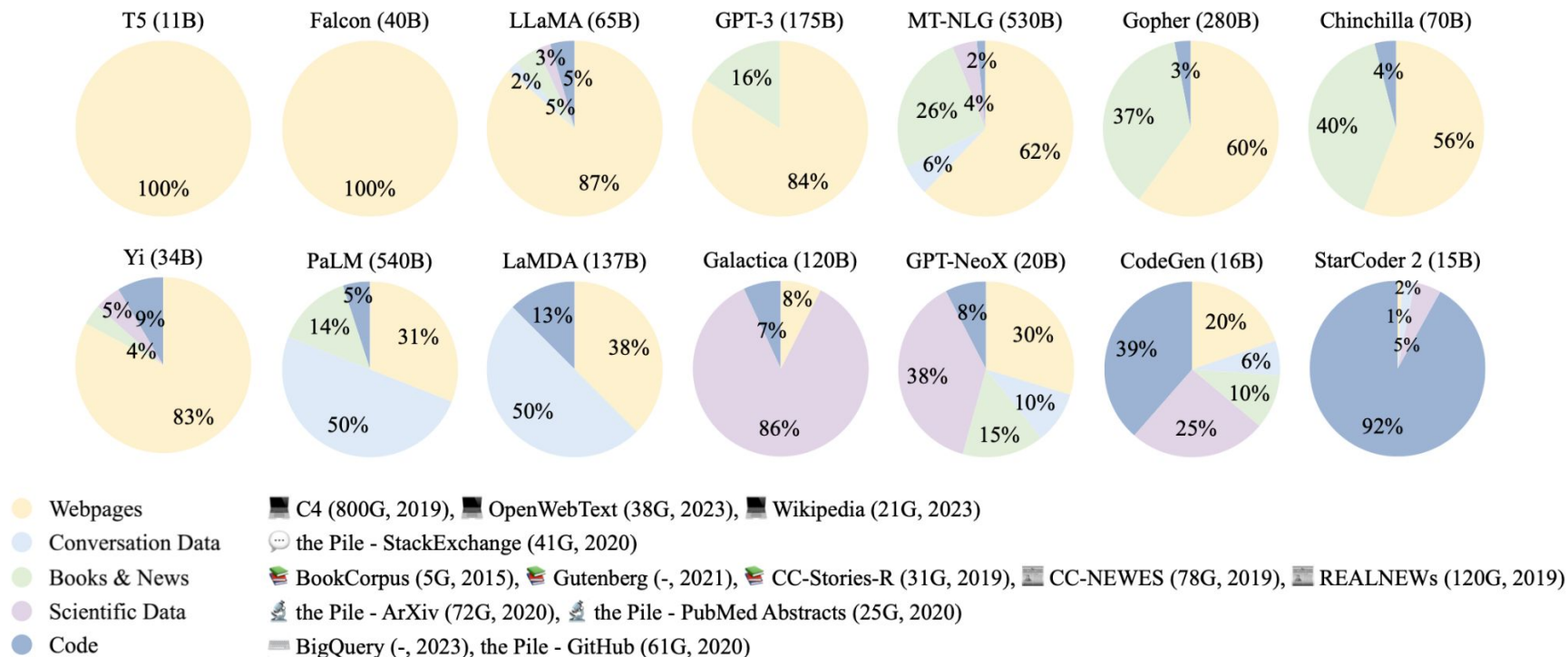
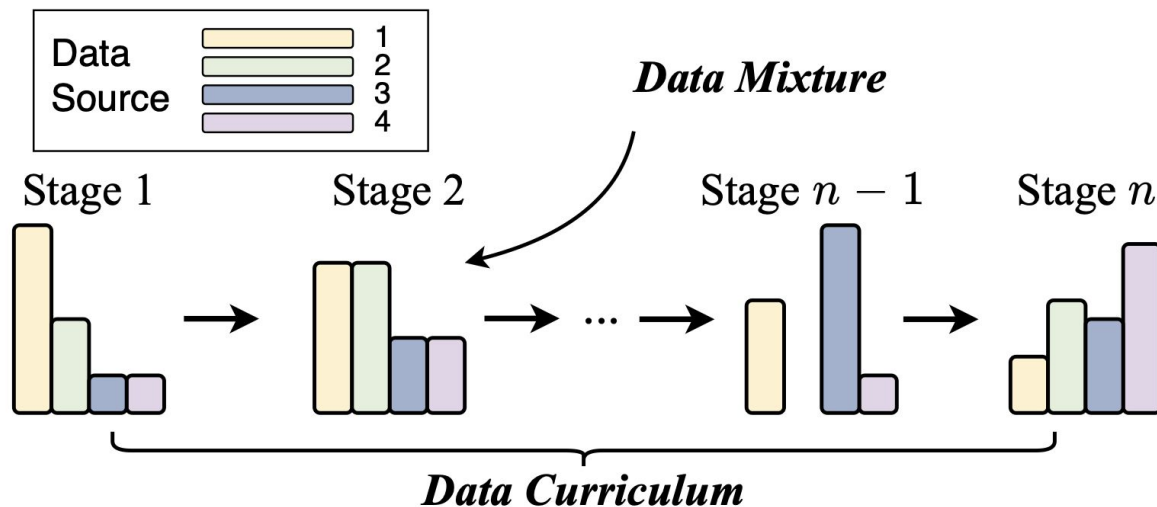


Figure 8 Data curation flow for pretraining data sources in DOLMA 3 MIX.

Data Mixtures



Data Curriculum



1) Pre-training

The LLM simply learns to continue the text instead of being conversational

Huge Text corpus



1
Train



2



What is an LLM?

3 Query



User

How do LLMs work?
What are LLM params?

4 Output

Notation



FLOPs

Floating-point ***O***perations

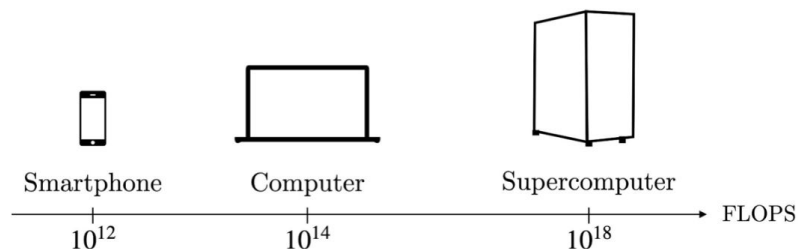
$f(\text{\#tokens}, \text{\#parameters})$



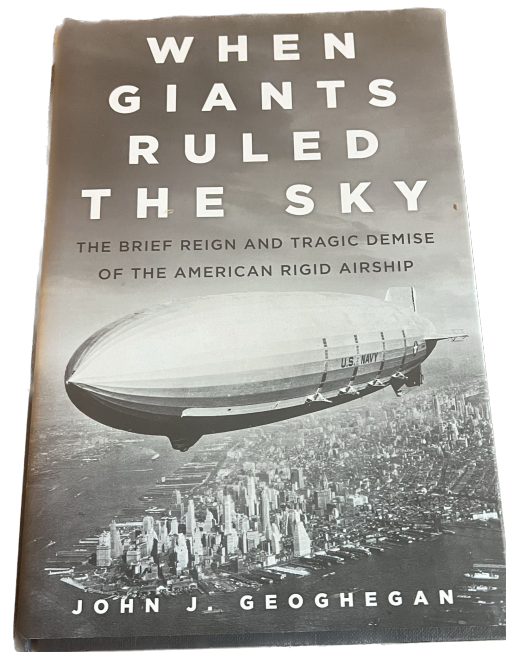
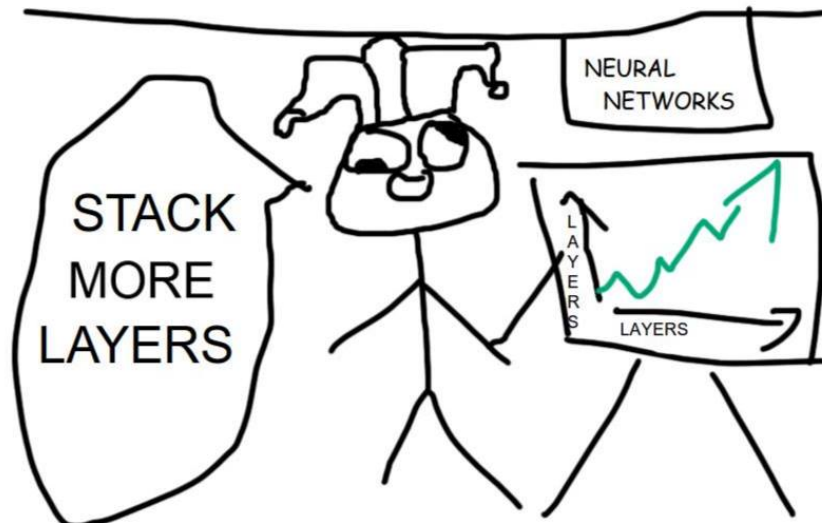
FLOPS or **FLOP/s**

Floating-point ***O***perations
per ***S***econd

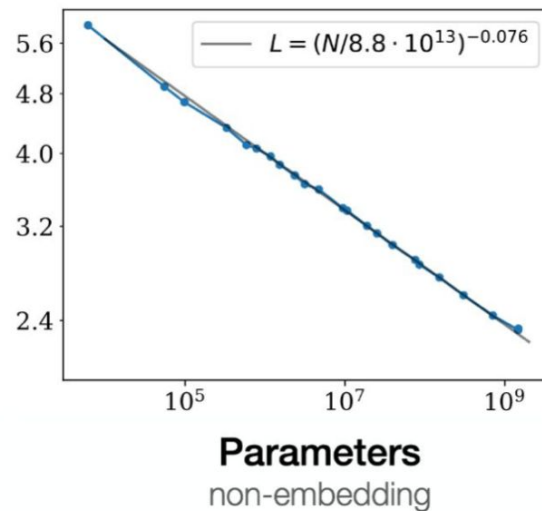
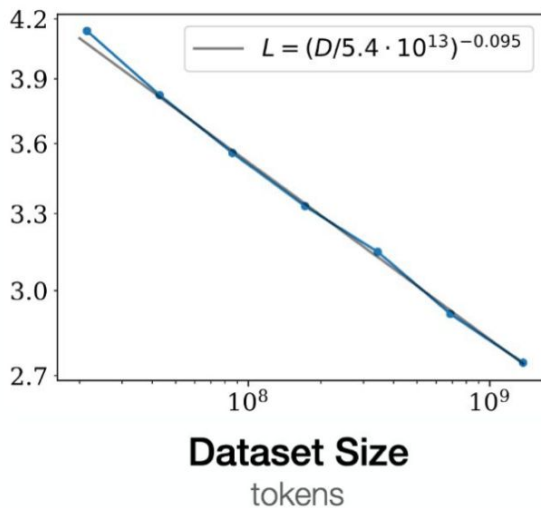
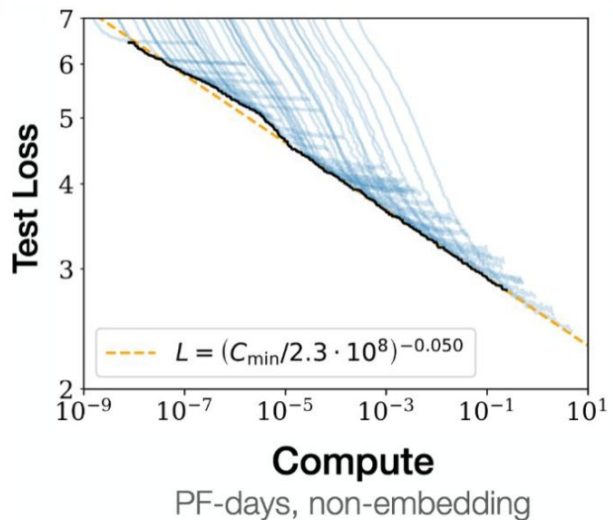
Computational performance



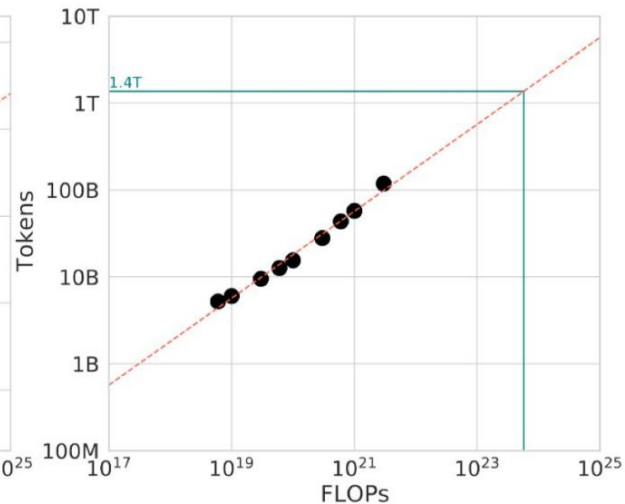
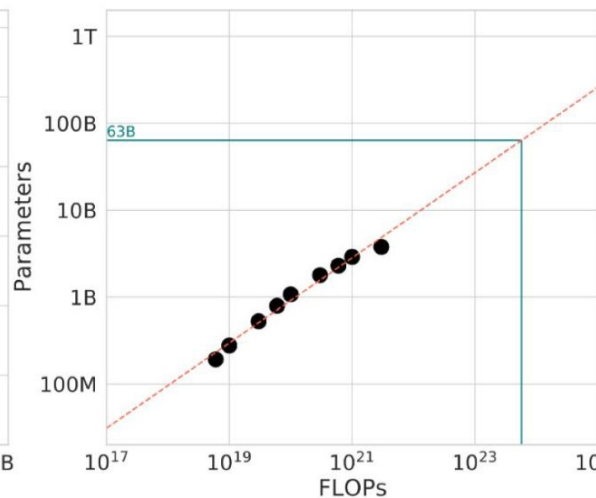
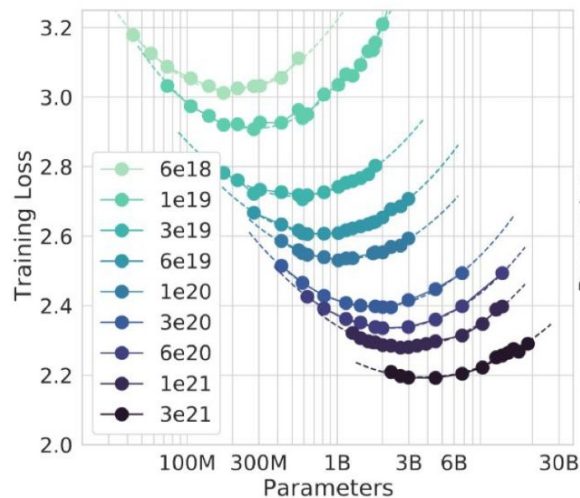
Pre-Training Recipe?



Pre-Training Scaling Laws (Kaplan 2020)



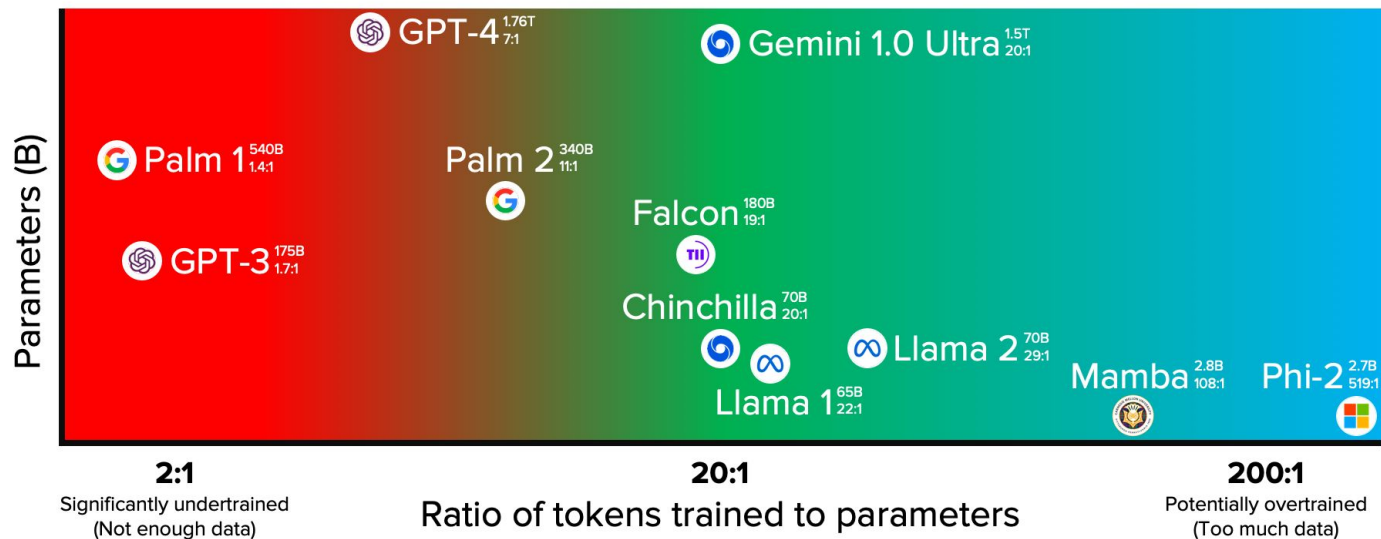
Pre-Training Scaling Law (Chinchilla 2022)



Pre-Training Scaling Law (Chinchilla 2022)

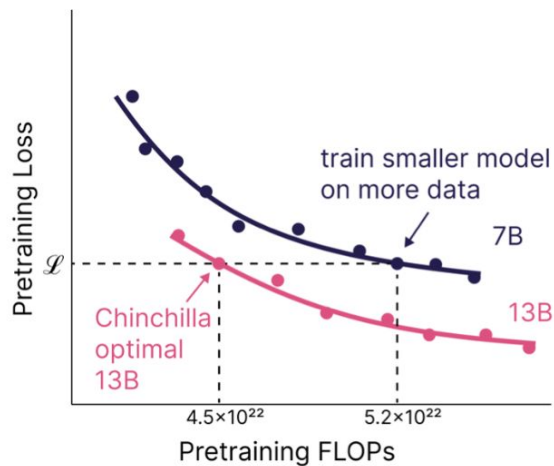
DATA-OPTIMAL (CHINCHILLA) MODEL HEATMAP

DEC/
2023

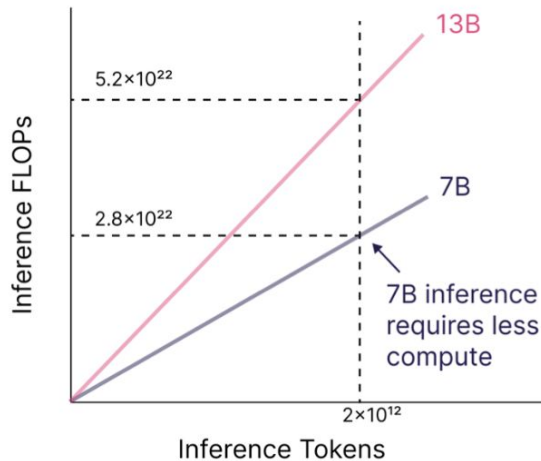


Selected highlights only. Mostly to scale. Informed estimates for Palm 2, GPT-4, and Gemini. Alan D. Thompson, November 2022, major update December 2023. <https://lilearchitect.ai/>

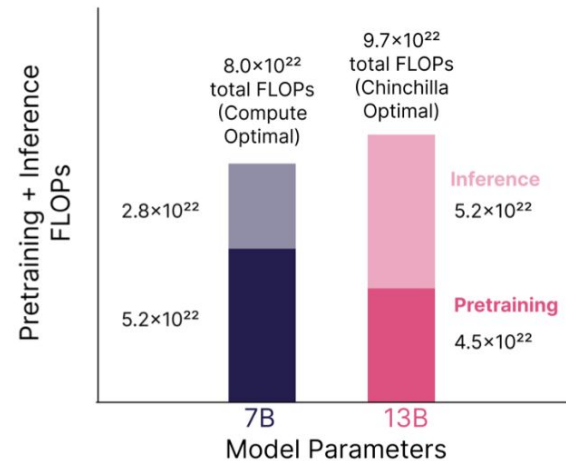
The Chinchilla Trap (2023)



(a)



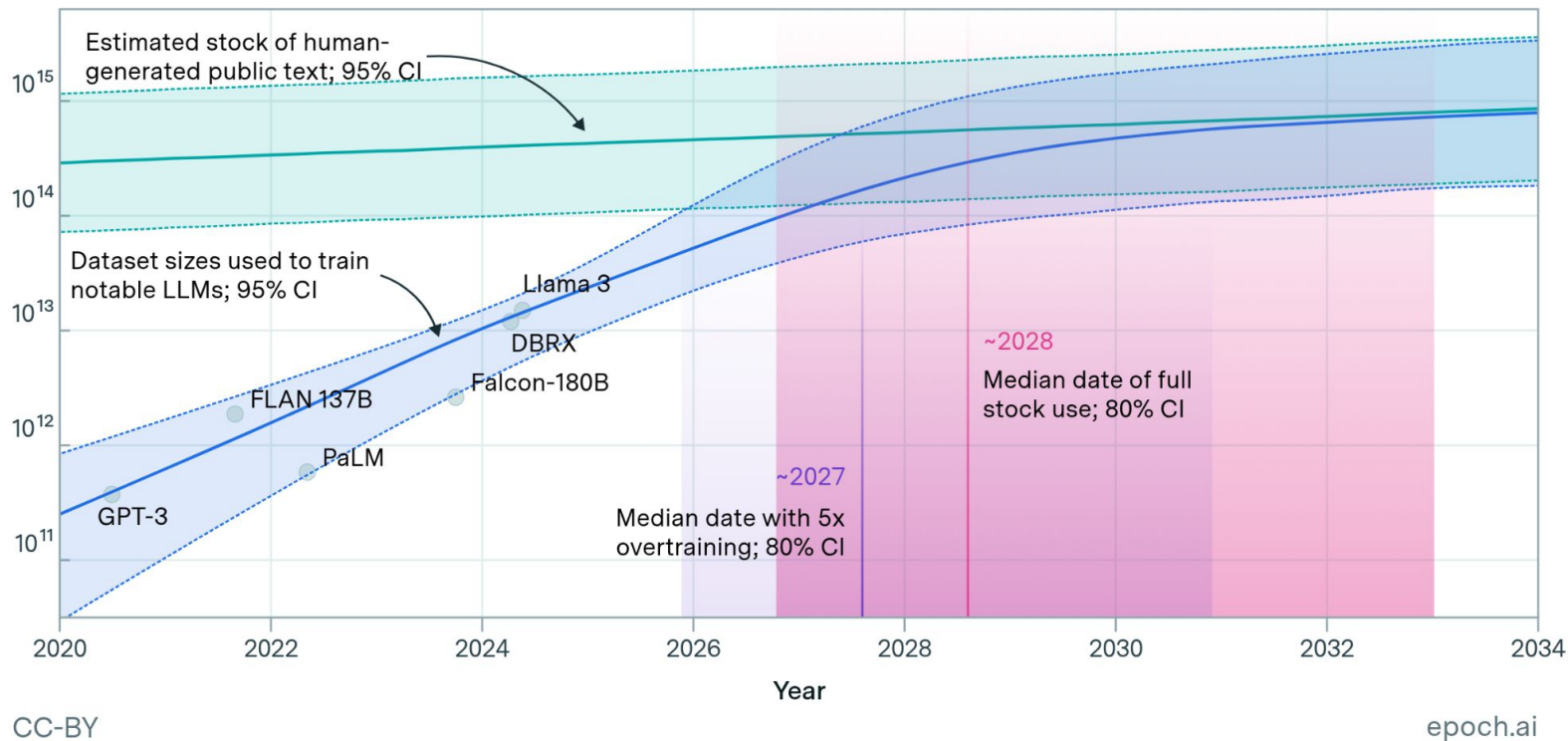
(b)



(c)

Projections of the stock of public text and data usage

Effective stock (number of tokens)



2) Instruction fine-tuning

The LLM becomes conversational,
providing helpful answers.

Instruction
response pairs



①
Train



②



What is an LLM?

③ Query



User

An LLM is a type of
ML model that trained...

④ Output

Instruction Tuning

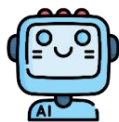
Effectiveness of Instruction Fine-Tuning

Before Instruction Fine-tuning

What is the capital of France?

Explain photosynthesis in one sentence.

Summarize this article.



What is the largest city in France?

Explain the water cycle in one sentence.

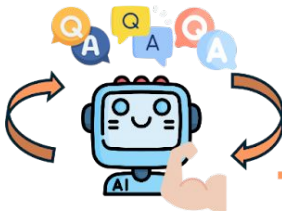
Summarize this text for me.

With Instruction Fine-tuning

What is the capital of France?

Explain photosynthesis in one sentence.

Summarize this article.



The capital of France is Paris.

Photosynthesis is the process by which plants convert sunlight into energy.

This article discusses the impact of climate change on global ecosystems.

Instruction Tuning

Instruction data

(Many tasks)

Instruction: "What are large language models?"

Task:

Question answering

Output: "Large language models (LLMs) are models that can generate human-like text by predicting the probability of a **word** given the previous words used in a sentence."

Instruction: "Rate this review"

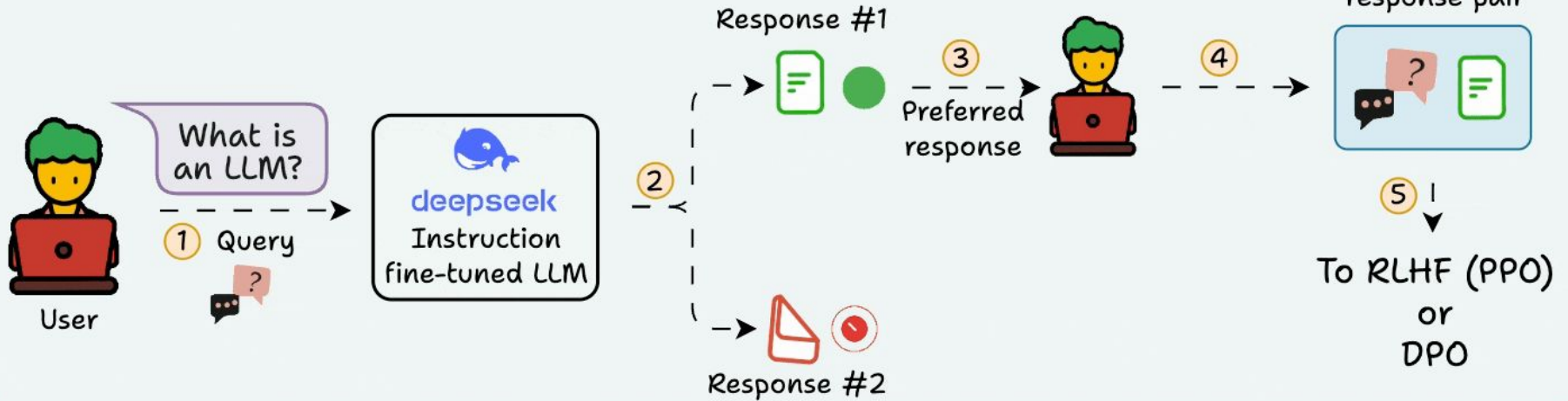
Task:

Sentiment analysis

Input: "This was a horrible place to eat!"

Output: "This is a **negative** review."

3) Preference Fine-tuning



Preference Tuning



You

precise name- what is it called in the api?

Which response do you prefer?

Your choice will help make ChatGPT better.



Response 1

In the OpenAI API, the model you are interacting with is specifically referred to as "gpt-4.0-turbo". This is the latest iteration of the Generative Pre-trained Transformer models developed by OpenAI, offering improved performance and capabilities compared to its predecessors. The "turbo" in the name indicates a more efficient and responsive version of the GPT-4 model, optimized for interactive applications like this one.



Response 2

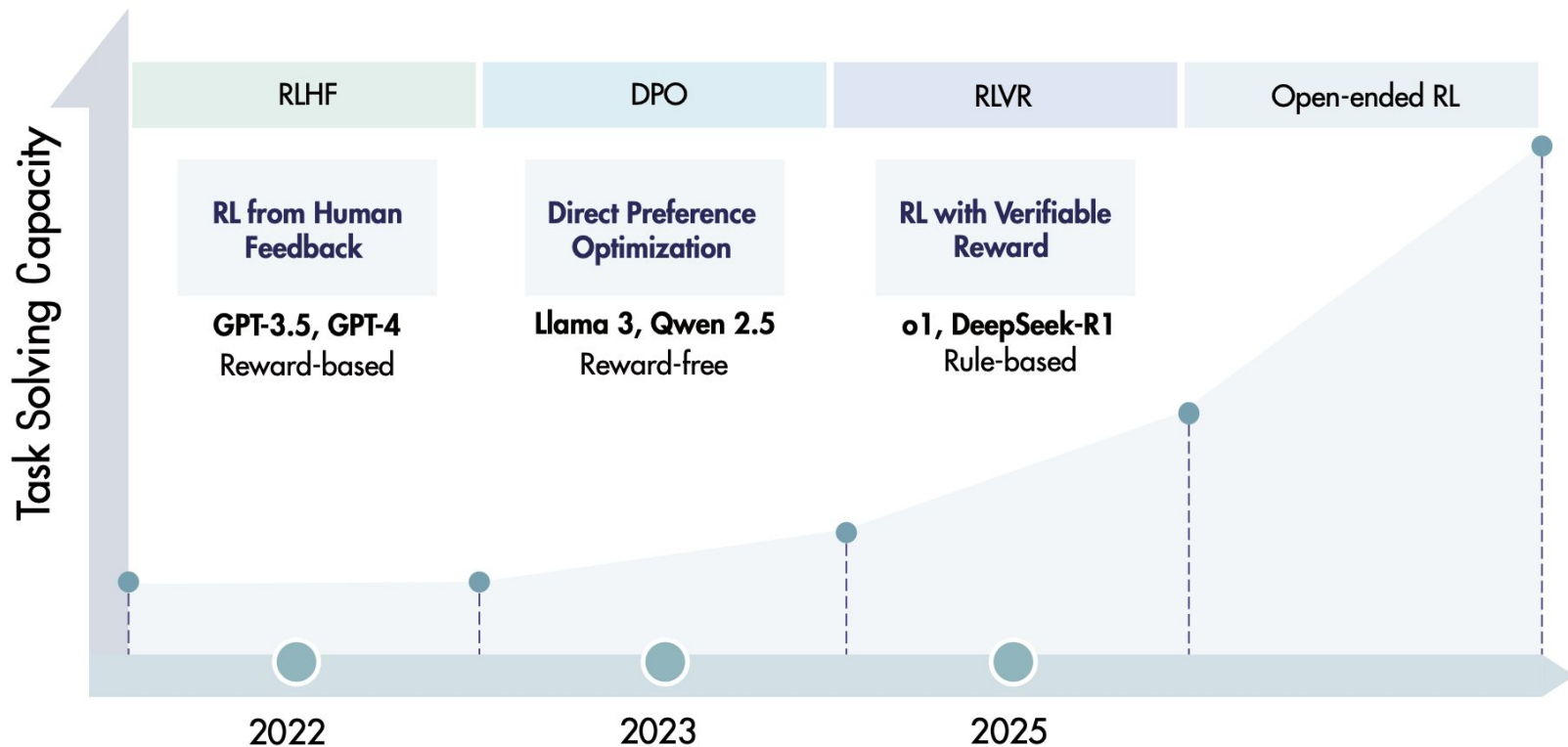
In the OpenAI API, the model you are interacting with is referred to as "gpt-4.0-turbo". This identifier is used to specify this particular version and iteration of the Generative Pre-trained Transformer model within the OpenAI API framework.



Message ChatGPT...



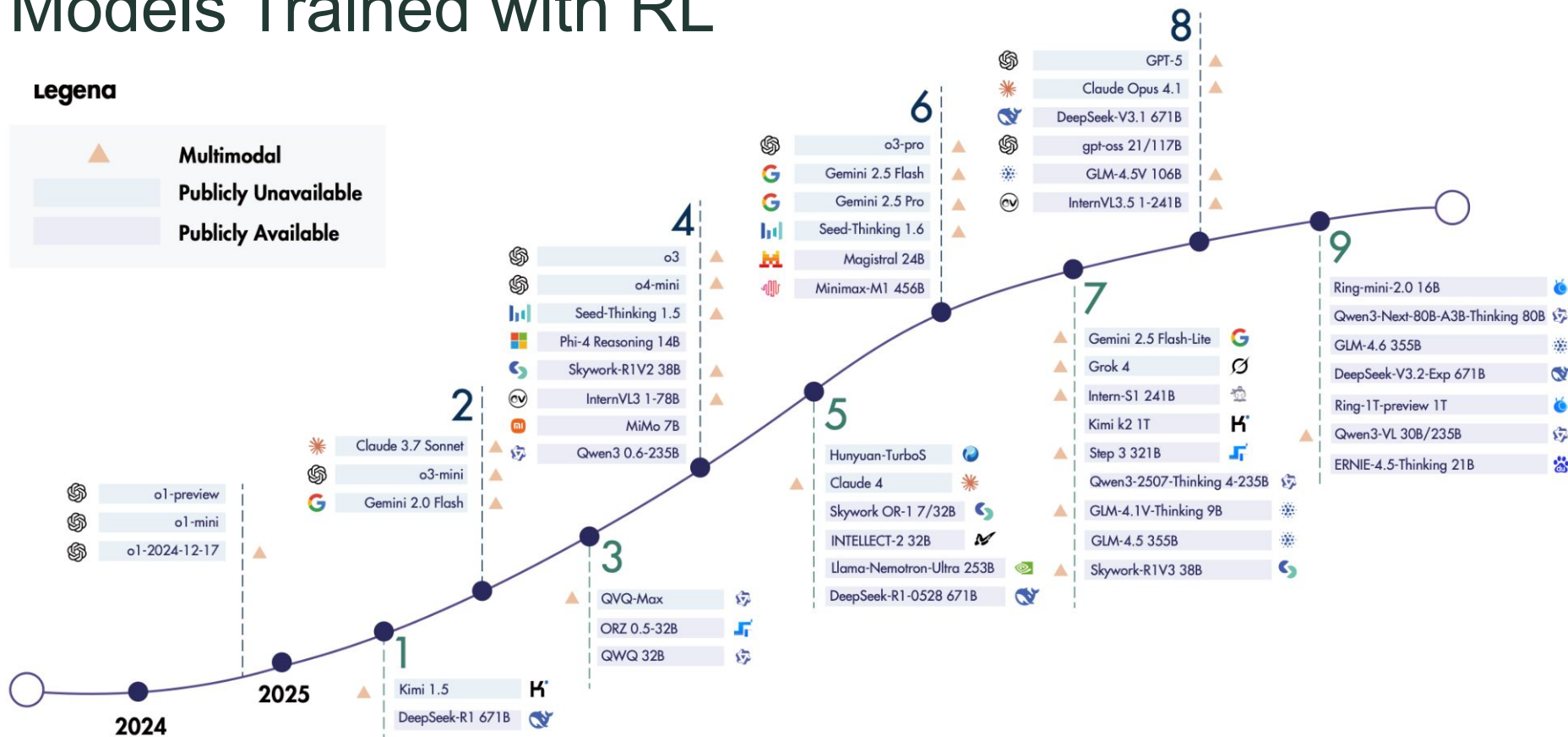
RL



Models Trained with RL

Legenda

▲	Multimodal
■	Publicly Unavailable
■	Publicly Available

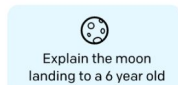


Preference Tuning (RLHF)

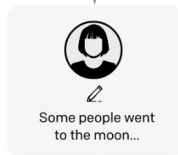
Step 1

**Collect demonstration data,
and train a supervised policy.**

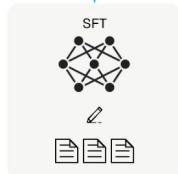
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



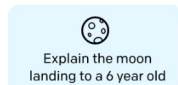
This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

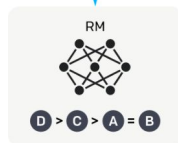
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



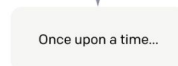
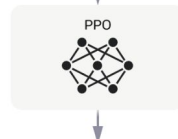
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

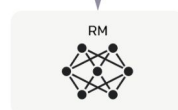
A new prompt
is sampled from
the dataset.



The policy
generates
an output.



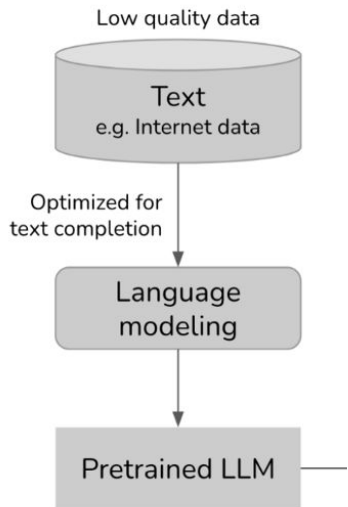
The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



Preference Tuning (RLHF)



Scale
May '23

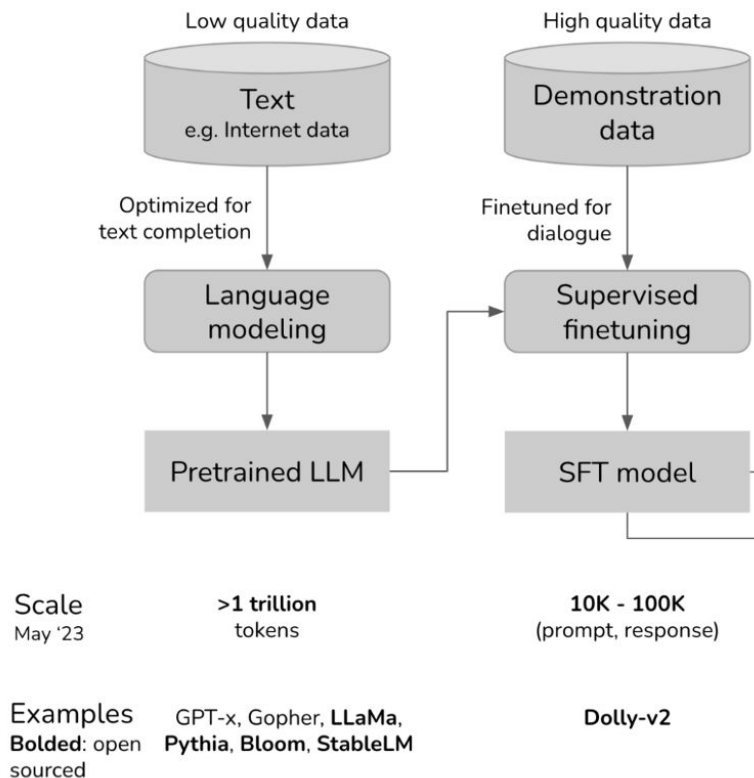
>1 trillion
tokens

Examples

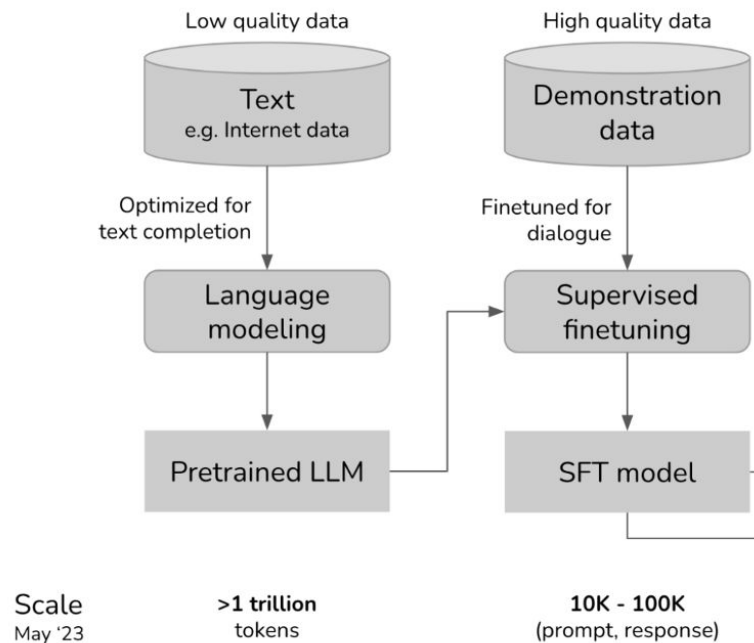
Bolded: open
sourced

GPT-x, Gopher, **LLaMa**,
Pythia, **Bloom**, **StableLM**

Preference Tuning (RLHF)



Preference Tuning (RLHF)

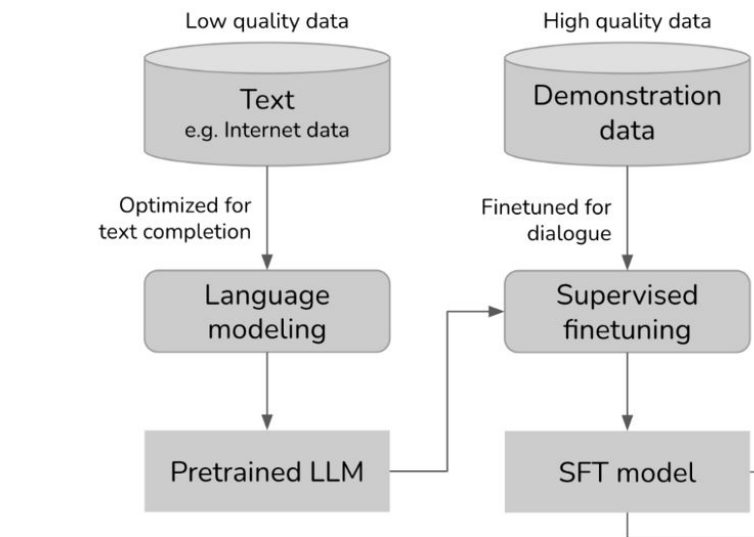


Examples
Bolded: open sourced
GPT-x, Gopher, **LLaMa**,
Pythia, **Bloom**, **StableLM**

Dolly-v2

	RedPajama	LLaMA*
CommonCrawl	878 billion	852 billion
C4	175 billion	190 billion
Github	59 billion	100 billion
Books	26 billion	25 billion
ArXiv	28 billion	33 billion
Wikipedia	24 billion	25 billion
StackExchange	20 billion	27 billion
Total	1.2 trillion	1.25 trillion

Preference Tuning (RLHF)



Scale
May '23

>1 trillion
tokens

10K - 100K
(prompt, response)

Examples GPT-x, Gopher, **LLaMa**,
Bolded: open **Pythia**, **Bloom**, **StableLM**
sourced

Dolly-v2

Table 1: Distribution of use case categories from our API prompt dataset.

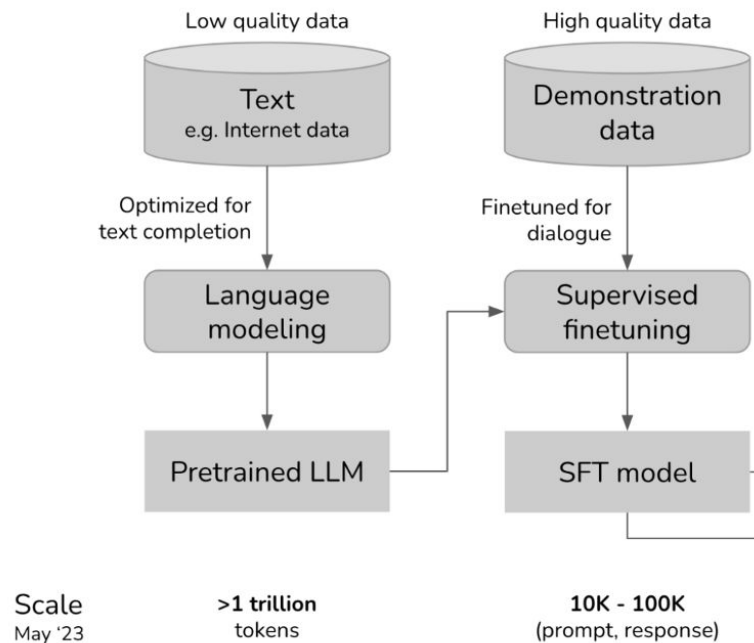
Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Table 2: Illustrative prompts from our API prompt dataset. These are fictional examples inspired by real usage—see more examples in Appendix A.2.1.

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: "" {summary} "" This is the outline of the commercial for that play: ""

The distribution of prompts used to finetune InstructGPT

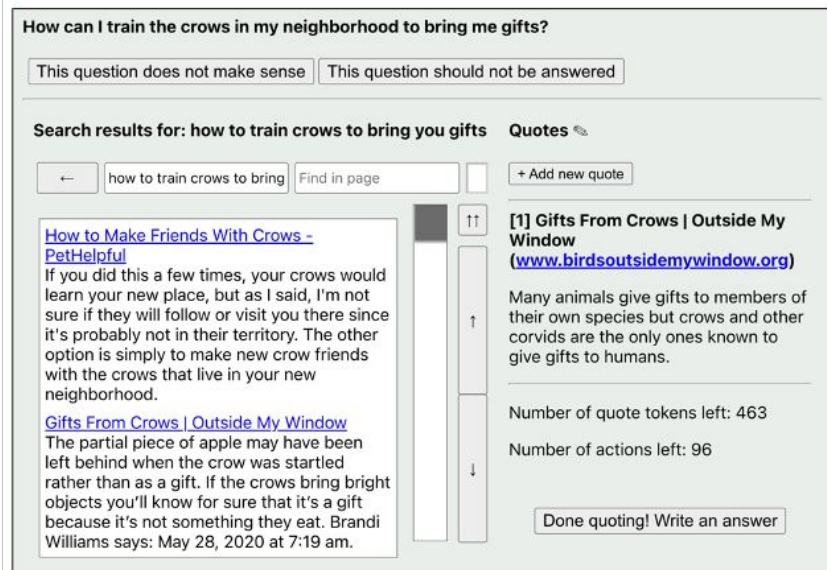
Preference Tuning (RLHF)



Examples

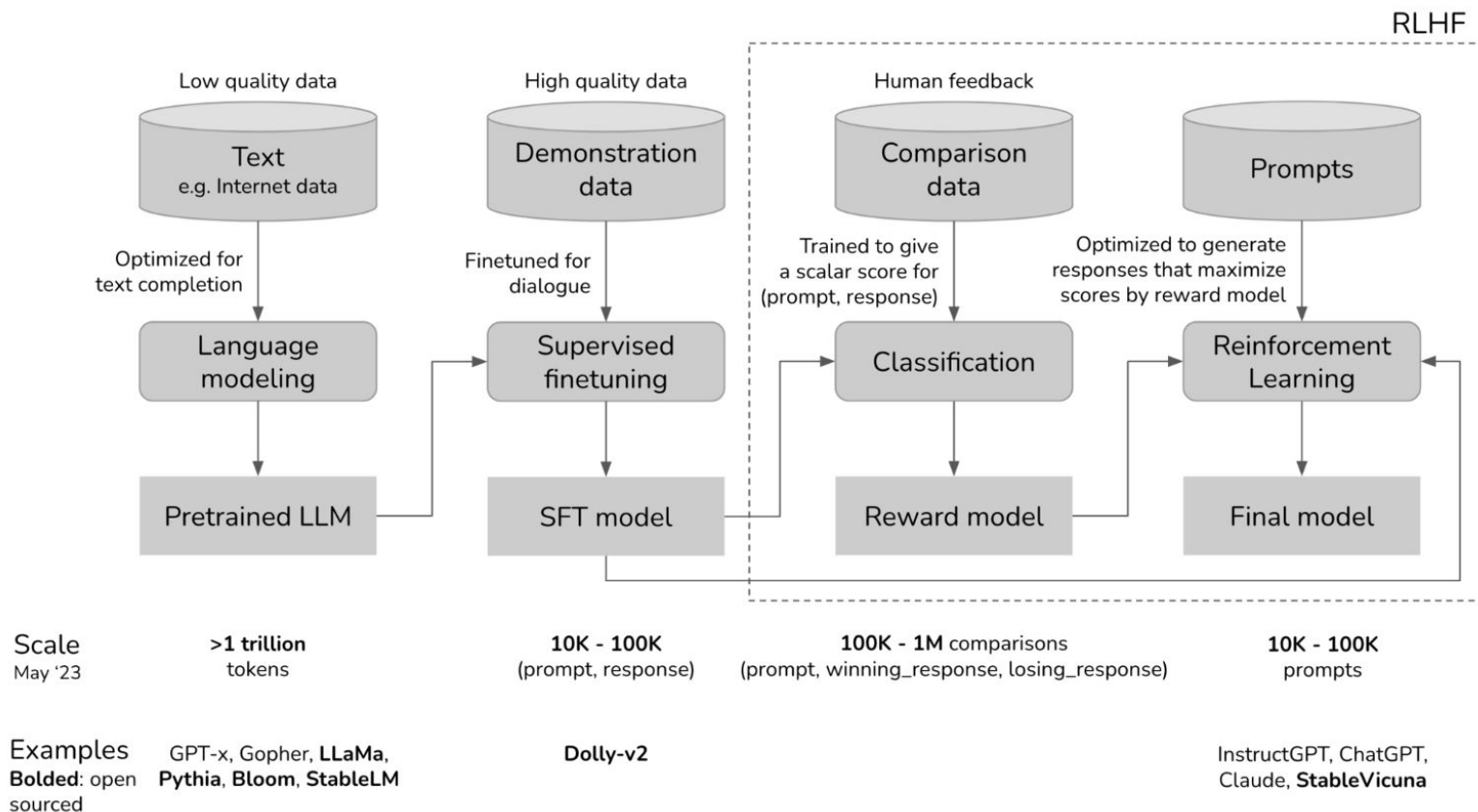
GPT-x, Gopher, **LLaMa**,
Bolded: open Pythia, Bloom, StableLM
sourced

Dolly-v2

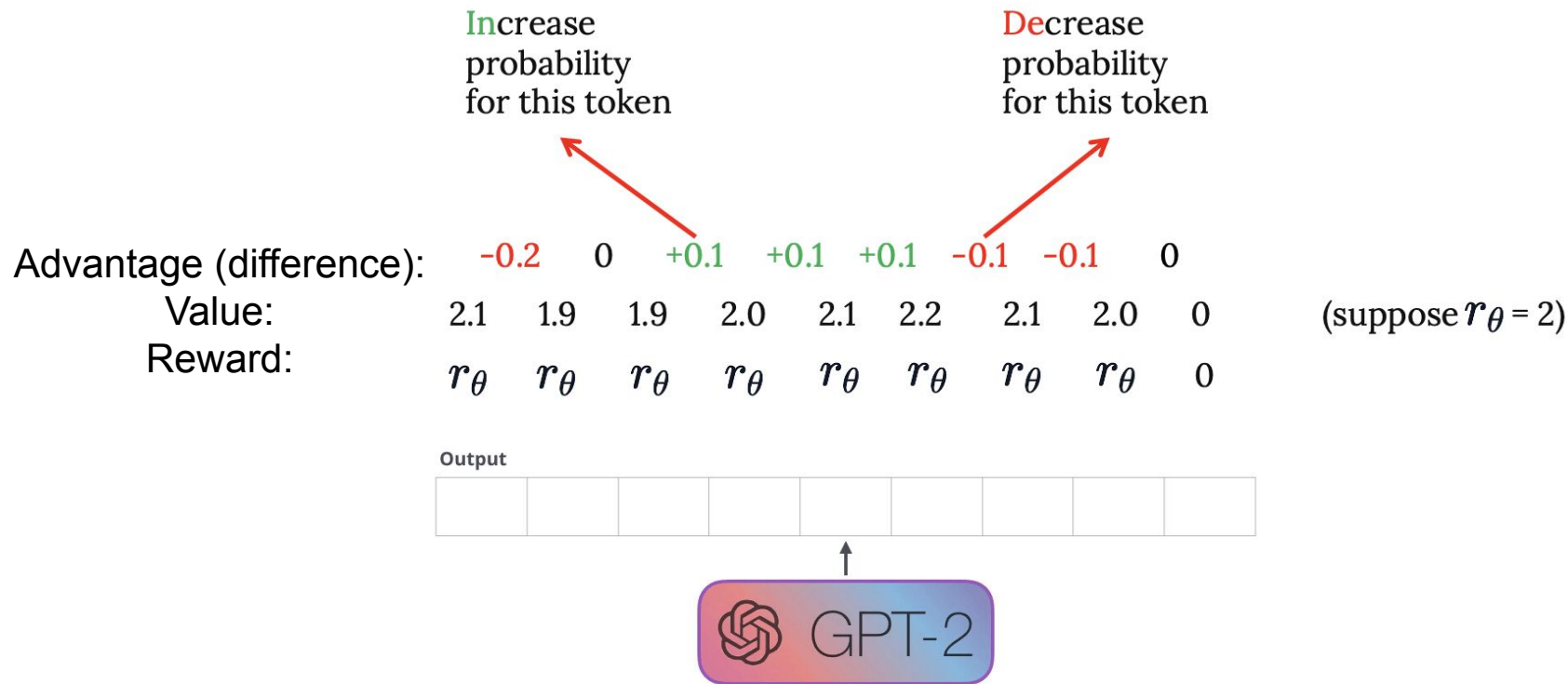


(a) Screenshot from the demonstration interface.

Preference Tuning (RLHF)

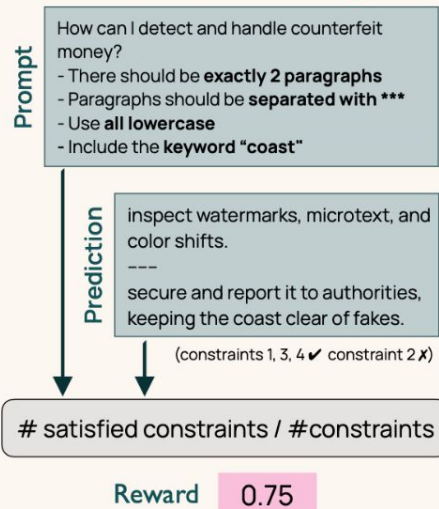


Preference Tuning (RLHF)

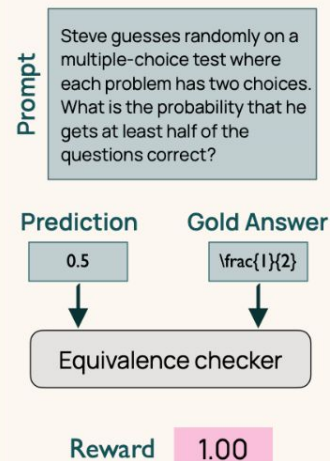


Verifiable Reward

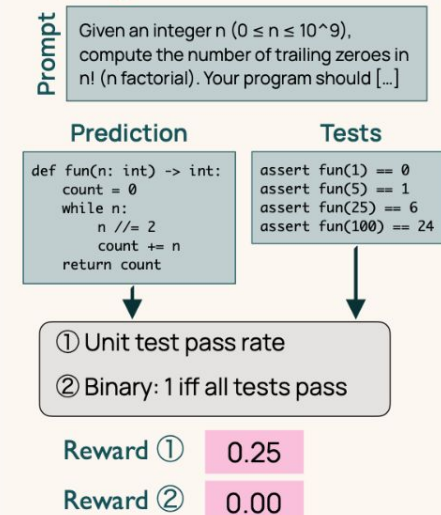
Instruction Following



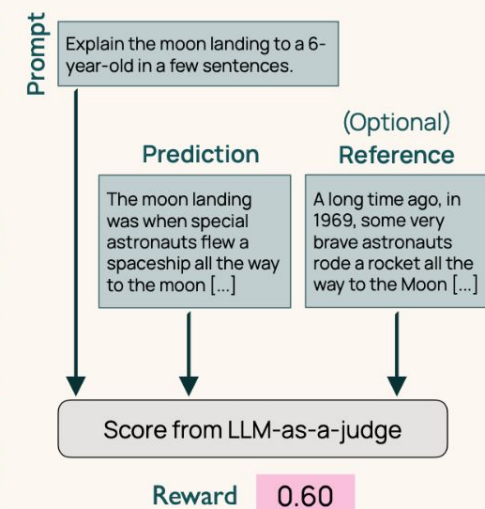
Math



Coding



General Chat



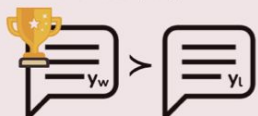
Verifiable Tasks

Non-Verifiable Tasks

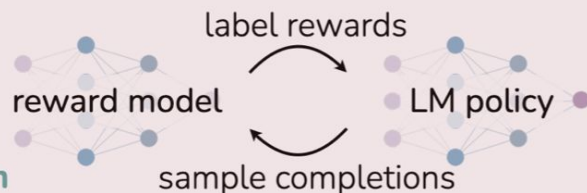
Preference Tuning (DPO)

Reinforcement Learning from Human Feedback (RLHF)

x: "write me a poem about
the history of jazz"



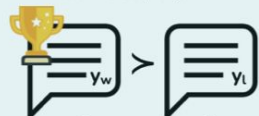
maximum
likelihood



reinforcement learning

Direct Preference Optimization (DPO)

x: "write me a poem about
the history of jazz"



maximum
likelihood



Why do we need RL?

- 1) Learning from negative feedback
- 2) RL forgets less
- 3) Better Generalization
- 4) Better Exploration



4) Reasoning fine-tuning

Reasoning task
with a definitive
answer



1

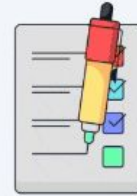


2



Reasoning-driven
response

3



Reward
calculation

4

Update model
params to
increase the
likelihood of
higher-reward
answers.

Chain-of-Thought (CoT)

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

CoT Trace

Problem:

James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?

Thinking Process

<think>

Okay, let me try to figure out this problem. So, James is writing a 3-page letter to two different friends twice a week. We need to find out how many pages he writes in a year. Hmm, let's break this down step by step.

First, let me understand the details. [...]

Wait, if he writes to 2 friends twice a week, does that mean he writes 2 letters each time, and he does this twice? [...] So per week, he writes $2 \text{ friends} * 3 \text{ pages} * 2 \text{ times}$. Let me check that. [...]

Alternatively, maybe it's 3-page letter to 2 different friends each time he writes, and he does that twice a week. So each time he writes, he's writing two letters (each 3 pages), so that's $2 * 3 = 6$ pages per writing session. Then, twice a week would be $6 * 2 = 12$ pages per week. Then, multiply that by the number of weeks in a year. But how many weeks are in a year? Typically, 52 weeks. So $12 \text{ pages/week} * 52 \text{ weeks} = 624$ pages. Is that right?

Wait, let me confirm. So, per week: [...] That makes sense.

But maybe I need to double-check if the problem is interpreted differently. [...] That seems consistent.

Wait, but wait a second. Is there any chance that the problem might be considering a year as 52 weeks, but some people might account for leap years or something else? But generally, in problems like this, unless specified, we take 52 weeks per year. So that's standard.

So, conclusion: [...] So the answer is 624. So I think that's correct.

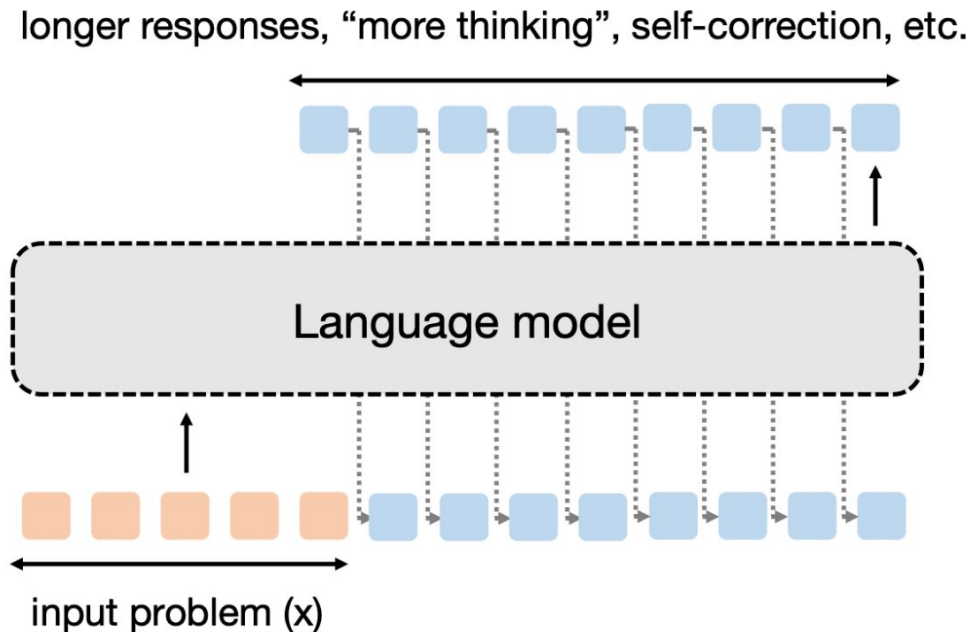
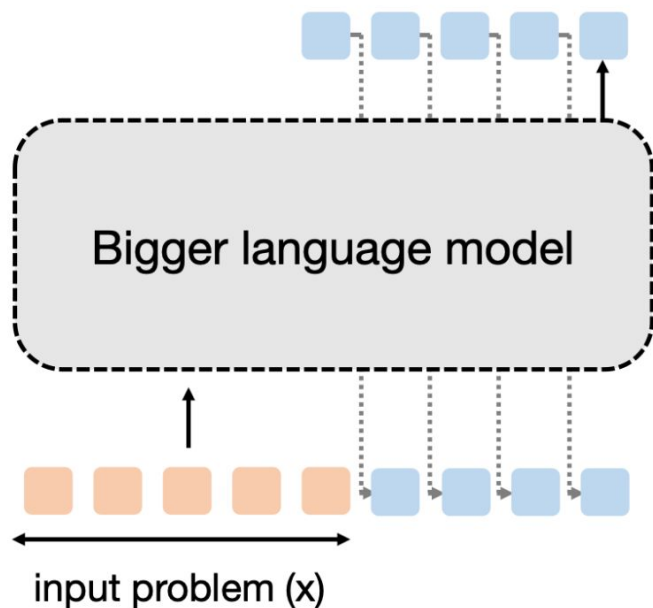
</think>

James writes 3-page letters to 2 friends twice a week. Here's the breakdown:
[...]

Answer: James writes **624 pages in a year.**

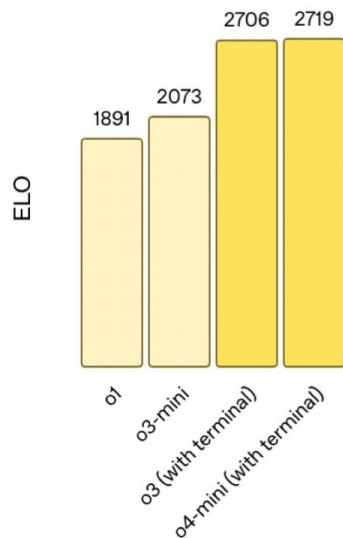
Answer

A new paradigm: test-time scaling

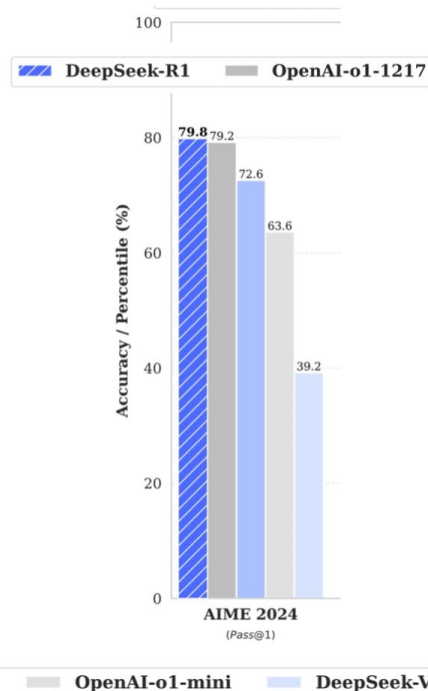
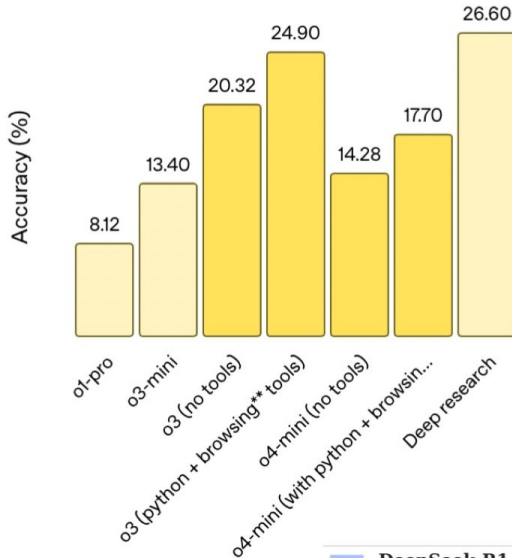


Thinking Models Top the Leaderboards!

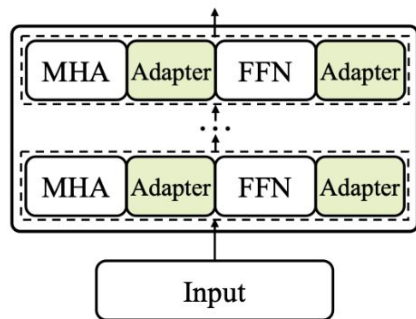
Codeforces
Competition Code



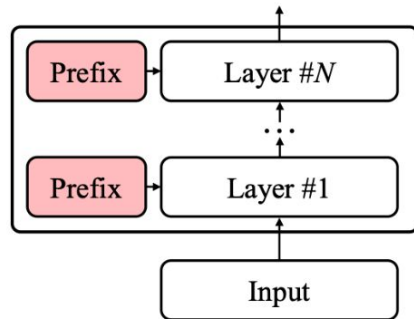
Humanity's Last Exam
Expert-Level Questions Across Subjects



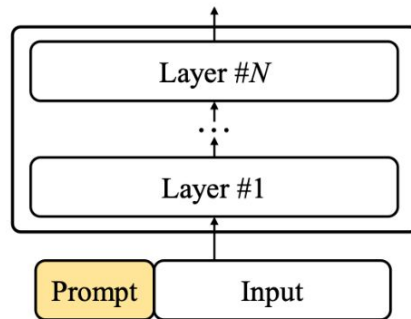
PEFT



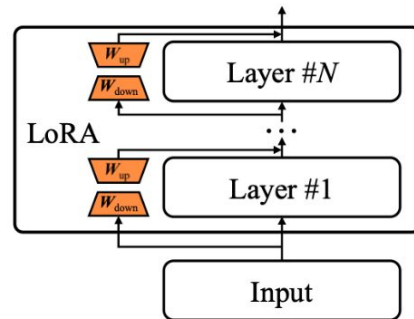
(a) Adapter Tuning



(b) Prefix Tuning

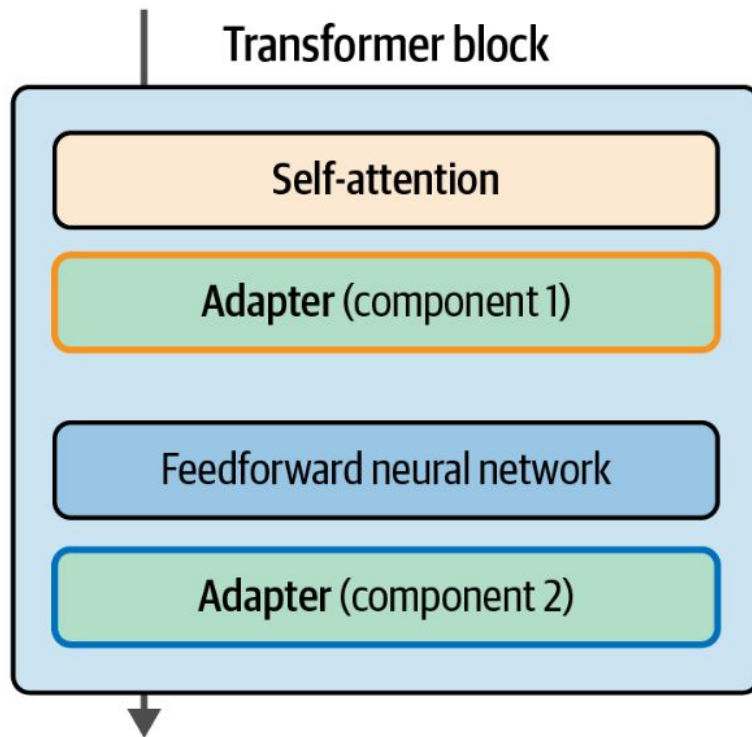


(c) Prompt Tuning

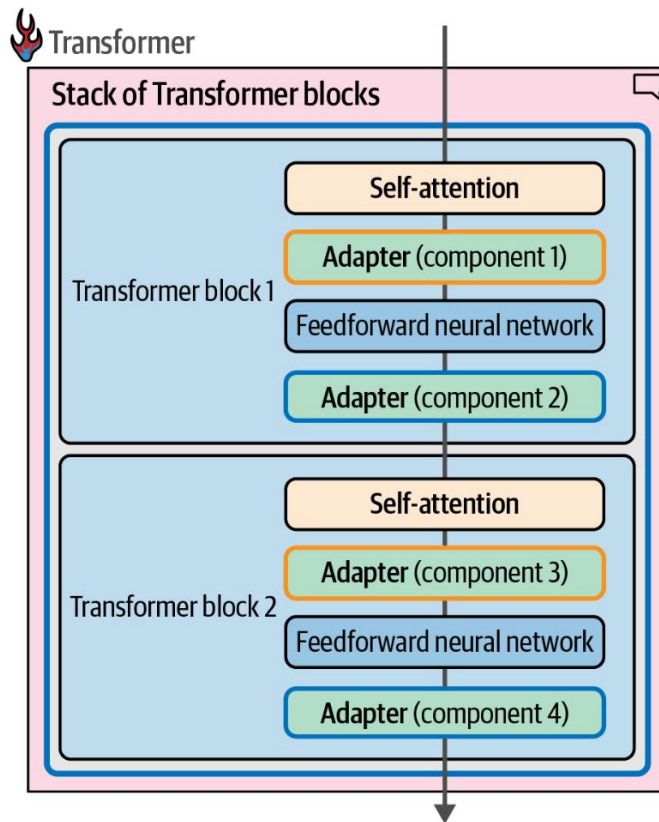


(d) Low-Rank Adaptation

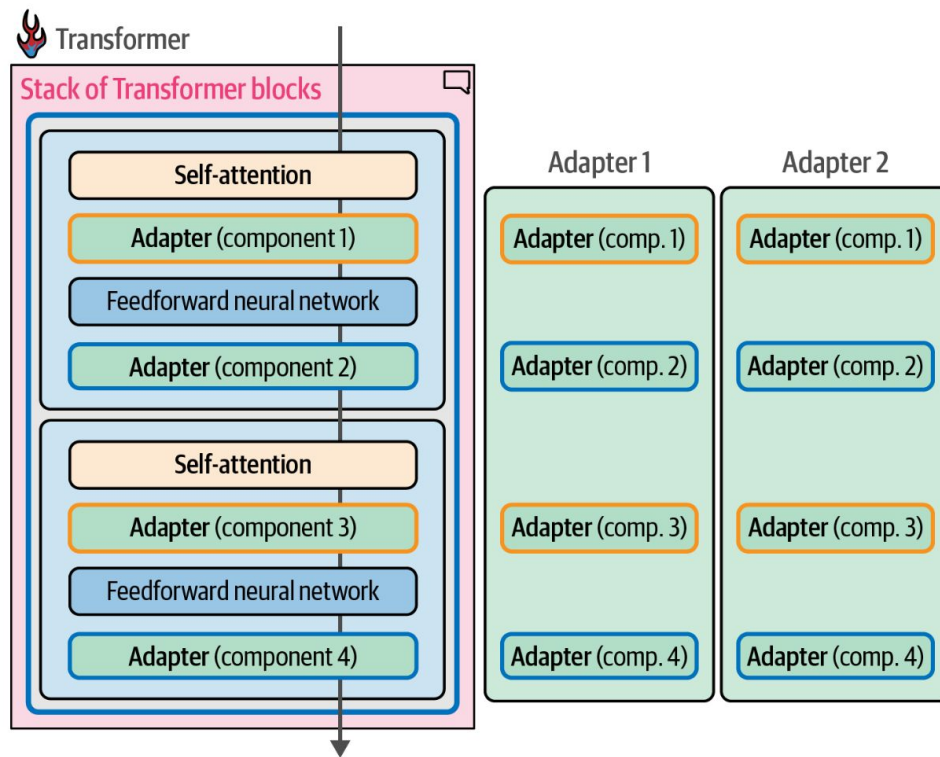
PEFT



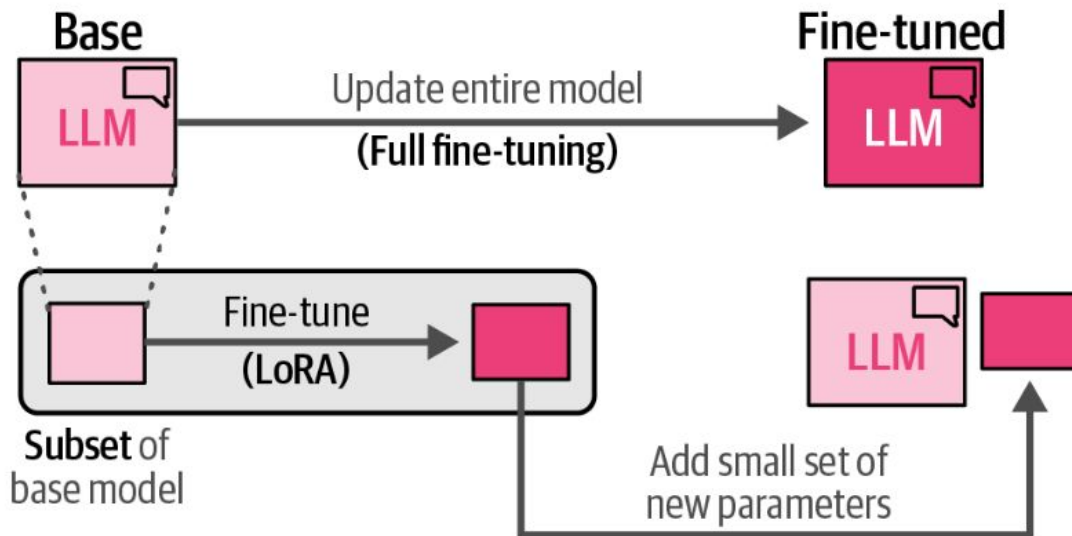
PEFT



PEFT



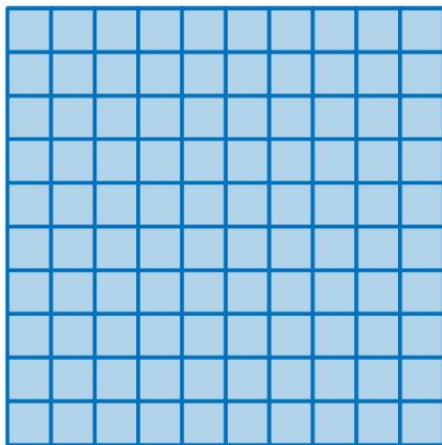
PEFT



LoRA

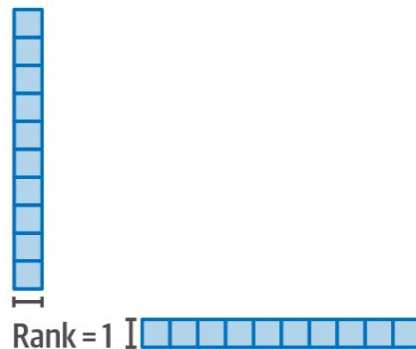
Weight matrix

Full rank (10×10)
Total parameters: 100



Low-rank weight matrix (rank = 1)

Total parameters: 20

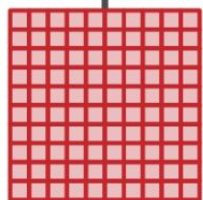


LoRA

Full fine-tuning

Update **all** weights

Input



Updated
During training



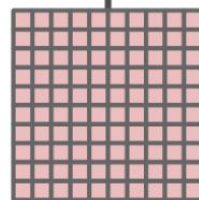
Low-rank adaptation

Update a **small representation**
of the weights

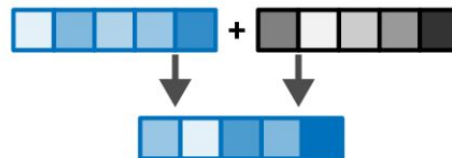
Input

Updated
During training

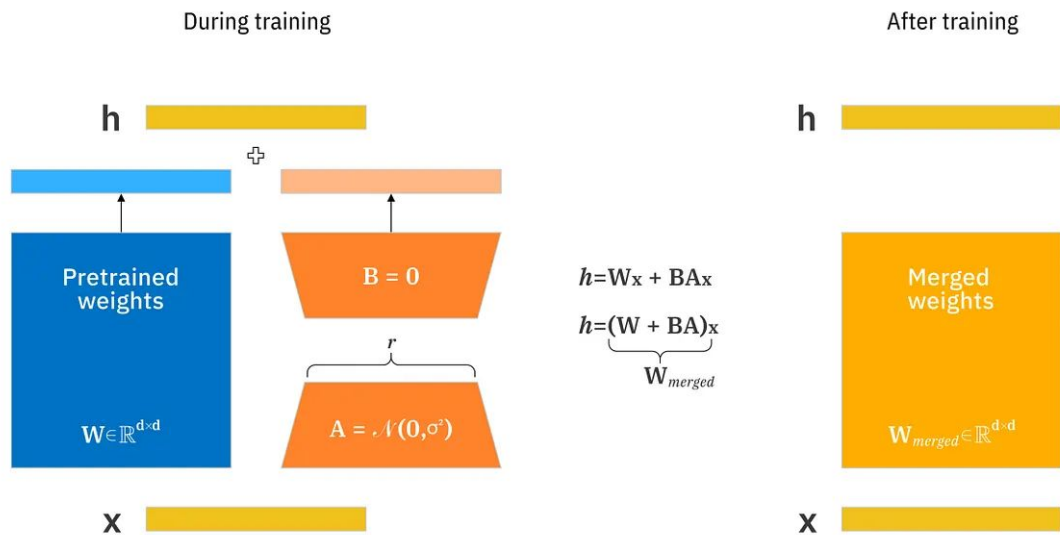
Rank = 2



Frozen
During training



LoRA



LLaMA family

